Xesta Documentation

Contents

Xesta..... 4
An adjusted Java Object / Xml Document Binding....4
Context.... 4

Getting Started with Xesta..... 4

Annotating Java Beans.....5

Managing namespaces.....9

Mapping collections of objects..... 11

Collection sorting strategies.....14

Bind Annotation Reference.....15

Repository types.....16

Consuming a remote RDF from the DBpedia with Xesta..... 17
Introducing the resource sample.... 17
Annotating the Pojo with mappings....18
Querying the resource....19
Querying different rdf resources with arq....19
References.... 19

Consuming a remote Xml file with Xesta..... 19
Introducing the resource sample.... 19
Annotating the Pojo with mappings....20
Querying the resource....21
References.... 21

Status..... 21

Changelog.....21

Acknowledgements.....22

References..... 22

Xesta

An adjusted Java Object / Xml Document Binding

Xesta is a java to xml binding tool. Xesta balances the approach between object and document side of the binding, trying to rely on tooling and practices aligned with each of the sides.

Context.

Document information is usually more complex, updated more frequently, including both data and structure and may be more focused on small details. Documents, Xml documents, are a great way of to handle detail and persistent, isolated (not directly backed by any system) data. They are also the backbone of the pulishing and digital humanities industry, including legal texts, books, product instructions, etc.

Xml Documents are usually explored with XPath indications. While Java comes by default with XPath 1.x the Xml community rely on newer versions. Xesta annotations are defined with XPath 3.x instead of positional indications. Future XPath versions may be adopted.

On the other side, Java and object oriented tools are tend to work with more structured data, less complex but with more volume. This part require more consistency and a more structured/less flexible approach. Java tooling frequently rely on Xml schemas when dealing with Xml documents. It is also common for Java tools to look for similar structures on both sides and rely on Xml for plain object serialization.

Xesta was presented as Fento on XML Prague 2026 and afterwards renamed as Xesta.

Getting Started with Xesta

Dowload and configure the Xesta jar and dependencies

Download the jar and dependencies or declare the dependency on the maven pom file.

Options:

- Declare the dependency on the pom file.

```
<dependency>
  <groupId>net.vionta.xml</groupId>
  <artifactId>xesta</artifactId>
  <version>1.0.1</version>
</dependency>
```

- If the version is not available on maven central you can download the last version from http://github.com/vionta/xesta/releases and install it locally.

Annotate your first class.

Xesta defines the xml document locations using Java annotations (Bind), the Xml locations are defined with the "expression" attribute.

Classes should implement the Serializable interface.

```
@Bind(expression = "//*:CstmrCdtTrfInitn" )
```

```
 public class CheckNoNS implements Serializable {
```

They should also have a no args constructor.

```
 public CheckNoNS() {}
```

Then you should annotate the fields.

```
  @Bind(expression = "*:PmtInf/*:DbtrAcct/*:Id/*:Othr/*:Id")
  String debtorAccount;
```

A complete Pojo example from a class used to retrieve the value attribute from a console appender from a log4j file.

```
 import java.io.Serializable;
 import net.vionta.xml.xesta.bind.annotation.Bind;

 @Bind(expression = "/*:configuration/appender[@name='CONSOLE']")
 public class LogLevelSimple implements Serializable {

  @Bind(expression = "param[@name='Threshold']/@value")
  private String level;

  public LogLevelSimple() {
    super();
  }

  public String getLevel() {
    return level;
  }

  public void setLevel(String level) {
   this.level = level;
  }

 }
```

Retrieve the document

In order to retrieve the document you use a repository. The simplest repository type is a File repository where you pass the file location.

```
  FilePathRepository repo = new FilePathRepository("./cases/iso20022/
GlobalPayment.xml");
  GlobalPayment globalPayment = new GlobalPayment();
  globalPayment = repo.load(globalPayment);
```

You can find more examples from the XmlPrague 2026 conference at http://www.github.com/vionta/xesta-samples.

Annotating Java Beans

The annotation process is intended to be as simple as possible. It is performed mostly with the Bind annotation, providing the XPath expressions and complementary configuration.

First Steps

Start by importing the Bind Class.

```
import net.vionta.xml.xesta.bind.annotation.Bind;
```

The next step is to provide an XPath expression for the main class.

```
@Bind(expression="*:appender")
public class Appender implements Serializable {
```

Each annotated class **MUST**:

- Implement the Serializable interface.

- Provide a no argument constructor

```
public Appender() {}
```

Annotating Simple Properties

Properties should be also marked with the Bind annotation.

```
@Bind(expression = "@*:name")
private String name;
@Bind(expression = "*:param[@name='Threshold']/@value")
private String level;
```

You can mark both elements, element text content or attributes. The data type is inferred from the Java type. By now the following Java types are supported:

- Integer

- Short

- Float

- Double

- Long

- Byte

For each property mapping regular setters and getters should be provided :

```
public String getName() {
 return name;
}

public void setName(String name) {
 this.name = name;
}
```

Providing Date Formats for dates.

Mapping netsted objects

Mapping nested objecs is quite similar to map regular properties. You just point the mapping expression to the Element that acts as the base context of the element properties.

In the following example we define a class (Location) with a nested "Group" Class. The Group class is mapped to an Xml "Band" element that has no namespace. Keep in mind that namespace declarations are not optional in XPath 3 and later.

```
@Bind(expression = "/concert-list/shows/Concert")
public class Location implements Serializable {

@Bind(expression = "./event/Band")
private Group group;

@Bind(expression = "./name")
private String name;
```

You must add the usual setters and getters and a no argument constructor. Also the element should implement the Serializable interface. If the Group class can be simply completed by adding the property mappings.

```
@Bind
public class Group implements Serializable {

@Bind(expression="./name")
private String name;

@Bind(expression = "./event")
private Event event;
```

Upward and downward element exploration

The class element mapping acts as a base level for the property mappings but properties are not limited to the element direct child elements. XPath navigation can be defined downward, upward or using axes and filters. This opens lots of possibilities on the read side but also faces some limitations when it comes to serialization, as the algorithm may not have enought information to determine where to put the information.

Downward examples

Let's start with an open office spreadsheet. We can map a Sheet class to the first spread sheet element of the file:

```
@Bind(expression = "//office:body/office:spreadsheet[1]",
 namespaceAlias = {"office"} ,
 namespaceUris = {"urn:oasis:names:tc:opendocument:xmlns:office:1.0"})
public class Sheet implements Serializable {

@Bind(namespaceAlias = {"table"} , namespaceUris =
{"urn:oasis:names:tc:opendocument:xmlns:table:1.0"})
private List Table> tables = new ArrayList Table>();
```

In an open office text document we could also map the font size to the fo:font-size attribute of the child text properties element:

```
@Bind(expression="./*:text-properties/@fo:font-size", namespaceAlias =
{"style","fo"},
```

```
namespaceUris = {"urn:oasis:names:tc:opendocument:xmlns:style:1.0",
"urn:oasis:names:tc:opendocument:xmlns:xsl-fo-compatible:1.0"})
private String fontSize;
```

When it comes to navigate backguards, think for example on a maven pom.xml file, we are explorating its contents like that:

```
@Bind(expression = "/project")
public class MavenProject implements Serializable {
 public MavenProject() {
 super();
 }
 @Bind(expression = "modelVersion")
 private String modelVersion;
 @Bind(expression = "groupId")
 private String groupId;
 @Bind(expression = "*:artifactId")
 private String artifactId;
 @Bind(expression = "*:packaging")
 private String packaging;
 @Bind(expression = "*:name")
 private String name;
 @Bind(expression = "version")
 private String version;
 @Bind(expression = "description")
 private String description;
 @Bind(expression="url")
 private String url;
 @Bind(expression = "./reporting/plugins/plugin", classNames =
"net.vionta.xml.xesta.test.beansamples.pom.Plugin")
 private ArrayList Plugin> plugins = new ArrayList Plugin>();
```

_ *NOTE : Mind that the example pom file does not have namespace declarations, as namespace references are not optional in XPath queries at 3.x version and following.*_

while we are mapping the collection of plugins, but for some reason, it may be convenient to get the project name (from an ancestor element) an place it in the child "plugin" class. In order to get the ancestor property "../" backguard operators are added to the XPath expressions.

```
@Bind
public class Plugin  implements Serializable {

@Bind(expression="artifactId")
private String id;

@Bind(expression="../../../name")
private String projectName;
```

Other exploration axes and operators, like "//" or ".//" could also be used.

XPath expressions in general can be used like conditions etc. This comes with a side effect on the serialization side, because a XPath query may be not concrete enough to determine where an element should be created or updated. We reject unspecific queries when it comes to object to Xml serialization. We can not allow to create an element like "// some-element" for example.

Managing namespaces

The first thing to underline is that, while in XPath 1.0 namespace declarations were optional, and XPath 1.0 is quite common in Java and web applications, they are not optional in modern versions of XPath. In order to avoid namespace declaration you can use the * operator to mark *:your-element-name now. A query like //the-element-name won't give you results with a document like:

```
<the-element-name xmlns="some-namespace" >
 <the-element-name>First non root element
   <the-element-name>Second non root element
   </the-element-name>
  </the-element-name>
  </the-element-name>
```

In contrast a query like /*/*:the-element-name will give you results on XPath 3.x and following. Also, if no namespace would be used on the document, a no namespace query on XPath 3.x will give you results.

Working with namespaces on Xesta

- **Documents without namespaces**. In this case you do not need to use wilcards on the mappings. Simple expressions, similar to the ones in XPath 1.0, would work:

```
   @Bind(expression = "/project")
   public class MavenProject implements Serializable {

  public MavenProject() {
   super();
  }

  @Bind(expression = "modelVersion")
  private String modelVersion;

  @Bind(expression = "groupId")
  private String groupId;

  @Bind(expression = "*:artifactId")
  private String artifactId;

   @Bind(expression = "./reporting/plugins", classNames =
"net.vionta.xml.xesta.test.beansamples.pom.both.Plugin")
  private ArrayList&amp;nbsp;Plugin> plugins = new ArrayList&amp;nbsp;Plugin>();
```

In this case we are using a maven document without namespaces like:

```
<?xml version="1.0" encoding="UTF-8"?>
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.some.library.group</groupId>
  <artifactId>somename</artifactId>
  <packaging>pom</packaging>
  <name>Some Project Name</name>
...
</project>
```

- Using wildcards is another common way to query documents with namespace declarations when we don't need anything specific about any of them. In the following example we will use it with a ISO2022 file.

```
 @Bind(expression = "//*:CstmrCdtTrfInitn" )
 public class CheckNoNS implements Serializable {

 public CheckNoNS() {}

 @Bind(expression = "*:PmtInf/*:PmtMtd")
 private String paymentMethod;

 @Bind(expression = "*:PmtInf/*:Dbtr/*:Nm")
 private String debtorName;

 @Bind(expression = "*:PmtInf/*:DbtrAcct/*:Id/*:Othr/*:Id")
 private String debtorAccount;
```

The sample file would start like this, and could be reviewed at https://developer.gs.com/docs/services/transaction-banking/iso20022samples/.

```
<?xml version="1.0" encoding="UTF-8" ?>
 <Document xmlns="urn:iso:std:iso:20022:tech:xsd:pain.001.001.03">
 <!-- Source : https://developer.gs.com/docs/services/transaction-banking/
iso20022samples/ -->
     <CstmrCdtTrfInitn>
         <GrpHdr>
             <MsgId>aeoueaoueaouoea</MsgId>
             <CreDtTm>2024-05-10T16:10:02.017+00:00</CreDtTm>
             <NbOfTxs>1</NbOfTxs>
             <CtrlSum>510.24</CtrlSum>
             <InitgPty>
                 <Id>
                     <OrgId>
                         <Othr>
                             <Id>Client-Id</Id>
```

- **Using Q{} syntax (not recommended).** One only namespace for line and element could be specified using Q{} syntax. Although simple, this approach is not recommended.

```
 @Bind(expression = "//
Q{urn:iso:std:iso:20022:tech:xsd:pain.001.001.03}CstmrCdtTrfInitn" )
 public class Check implements Serializable {

  public Check() {}

  @Bind(expression =
 "Q{urn:iso:std:iso:20022:tech:xsd:pain.001.001.03}PmtInf/*:PmtMtd")
  private String paymentMethod;

   @Bind(expression =
 "Q{urn:iso:std:iso:20022:tech:xsd:pain.001.00103}PmtInf/*:Dbtr/*:Nm")
  private String debtorName;
 ...
```

- **Using namespace alias and uris.** This is the most complete and recommended way if namespaces are needed and wildcards are not enough.

```
@Bind(expression = "//iso:CstmrCdtTrfInitn" , namespaceAlias = {"iso"} ,
namespaceUris = {"urn:iso:std:iso:20022:tech:xsd:pain.001.001.03"})
public class CheckNS implements Serializable {

 public CheckNS() {}

 @Bind(expression = "iso:PmtInf/iso:PmtMtd" , namespaceAlias = {"iso"} ,
namespaceUris = {"urn:iso:std:iso:20022:tech:xsd:pain.001.001.03"})
 private String paymentMethod;

 @Bind(expression = "*:PmtInf/*:Dbtr/*:Nm")
 private String debtorName;

  @Bind(expression = "*:PmtInf/*:DbtrAcct/*:Id/*:Othr/*:Id")
  private String debtorAccount;
```

More than one namespace could be added, like you could see in the following example (styles list).

```
@Bind(expression="office:document-styles", namespaceAlias =
{"office"} ,namespaceUris =
{"urn:oasis:names:tc:opendocument:xmlns:office:1.0"})
public class Document implements Serializable {

   @Bind(expression="office:styles", namespaceAlias = {"office", "style"},

 namespaceUris = {"urn:oasis:names:tc:opendocument:xmlns:office:1.0",
   "urn:oasis:names:tc:opendocument:xmlns:style:1.0"})
 private List Style> styles = new ArrayList Style>();
```

Another example could be checked at the following property

```
 @Bind(expression="./style:text-properties/@fo:font-
size", namespaceAlias = {"style","fo"}, namespaceUris
 = {"urn:oasis:names:tc:opendocument:xmlns:style:1.0",
 "urn:oasis:names:tc:opendocument:xmlns:xsl-fo-compatible:1.0"})
  private String fontSize;
```

Like in any XPath query, wildcards and namespaces can be mixed

```
@Bind(expression="./*:text-properties/@fo:font-size",
namespaceAlias = {"style","fo"}, namespaceUris =
{"urn:oasis:names:tc:opendocument:xmlns:style:1.0",
"urn:oasis:names:tc:opendocument:xmlns:xsl-fo-compatible:1.0"})
 private String fontSize;
```

Mapping collections of objects

Xml usually hold collections of elements, repetitions of elements that we would naturally bind to java.util.List subclasses.

Single and multiple collections is an important concept for Xesta. Programming approaches tend to organize collections of similar elements in its own separated bags. This is not mandatory in Java, a subinstance of List can hold instances of different objects. This is not the usual practice. Generics and other bind or serialization tools tend to the "one bag for each object type" practice.

In Xml, the element name and namespace is a primary source of information and as a consequence mixed types of elements and the ordering may be really significative.

We call single collection mapping to the collections where we have only one type of child on the Java side, and multiple collections where we may map more than one object type on a java side for the same collection.

Mind that using XPath we may wrap child elements from more than one element in a single Java collection and we could also take single types of java objects from an element with different child nodes. This is one of the main appealings from this technique, although it comes with a downside when we switch to serialzing.

Single Collections

Mapping a single collection is quite simple, and support several combinations:

Mapping the element name on the child side. This is probably one of the most intuitive ways. In this case we explore a Log Level document, the appender collection points the context of the element collection to the base of the document content ("../*"). There the class type guess can be taken from the List generics declaration.

```
@Bind(expression = "/*:configuration")
public class LogLevel implements Serializable {

@Bind(expression = "../*")
private ArrayList Appender> appender = new ArrayList Appender>();
```

In the Child Element class, we would be considering all the "appender" elements with any schema.

```
@Bind(expression="*:appender")
public class Appender implements Serializable {

@Bind(expression = "@*:name",key=true)
private String name;

@Bind(expression = "*:param[@name='Threshold']/@value")
private String level;
```

In this case, the context is pointed in the collection part and the element type on the child side.

- **Mapping the child element on the collection side**. This is also a very simple approach. In the following example we would be defining the full route of a maven plugin element on the collection side. No separated context and element.

```
@Bind(expression = "/project")
public class MavenProject implements Serializable {

@Bind(expression = "*:artifactId")
private String artifactId;

@Bind(expression = "./reporting/plugins/plugin")
private ArrayList Plugin> plugins = new ArrayList Plugin>();
```

On the child side we would not need to add anything at the class level, as the class type is especified using generics.

```
public class Plugin  implements Serializable {
```

```
@Bind(expression="artifactId")
private String id;
```

- **Mapping the element name on the child side**. In this case we would have an annotation on the list element that states the context of the collection.

```
@Bind(expression = "../*")
private ArrayList Appender> appender = new ArrayList Appender>();
```

On the child element side we would especify the element that should be binded to the pojo.

```
@Bind(expression="*:appender")
public class Appender implements Serializable {

@Bind(expression = "@*:name",key=true)
private String name;

@Bind(expression = "*:param[@name='Threshold']/@value")
private String level;
```

The key attribute of the annotation states that the property value could be used as an identifier of the element. This is used in collections where we want to take into consideration moving elements, sorting, etc.

- **Providing the child class element**. This is another possibility, reserved usually for the multielement collections, where we state the class names that should be inspected for element mappings.

```
@Bind(expression = "/*:configuration")
public class LogLevel implements Serializable {

@Bind(expression = "../*", classNames =
"net.vionta.xml.xesta.test.beansamples.log.Appender")
private ArrayList Appender> appender = new ArrayList Appender>();
```

Multiple Collections

Multiple collections are list of elements that may have more than one element child type. This is a less used practice on the Java enterprise but may make a lot of sense when working with Xml documents. Xml Element semantic and metadata capabilities enables some of the most complex and flexible use cases on data management, and the element name, schema, etc. may be used to carry significant information.

Multiple collections require that you add the child element class names to the list declaration

```
@Bind(expression="./*:*",
classNames = {
"net.vionta.xml.xesta.test.beansamples.list.salvora.neu1.Parameter.class",
"net.vionta.xml.xesta.test.beansamples.list.salvora.neu1.RequestParameter.class",

"net.vionta.xml.xesta.test.beansamples.list.salvora.neu1.PathParameter.class"})
```

```
private ArrayList ? extends Serializable> parameters = new
ArrayList Serializable>();
```

Where in the child objects the class mapping should state the Xml element to be mapped to each one.

```
@Bind(expression="/*:parameter")
public class Parameter implements Serializable {

public Parameter() {}

@Bind(expression="@key")
private String key;

@Bind(expression="@value")
private String value;
```

Collection sorting strategies

The object binding technique is actually limiting. The possibilities of what we could do or the way that we may interact with an Xml file is almost endless. Annotation strategies are small properties added to the mapping annotation in order to define specific behaviours.

Element identifying and sorting

- Bind By Key (BIND_COLLECTION_BY_KEY)

In this case, the collection elements should have a mapped property that uniquely identifies the elements on the Xml side. This key is used to add, remove or sort the elements before serializing. This is the default case if a key attribute is defined.

```
  @Bind(expression = "./reporting/plugins",
classNames = "net.vionta.xml.xesta.test.beansamples.pom.both.Plugin")

private ArrayList Plugin> plugins = new ArrayList Plugin>();
```

In this case the child element has a key attribute.

```
@Bind(expression = "plugin")
public class Plugin  implements Serializable {

@Bind(expression="artifactId", key = true)
private String id;
```

- Bind Collection by positon (BIND_COLLECTION_BY_POSITION).

In this case we assume that elements can't or should not be identified and possition should be used instead.

Bind by position is assumed when no key attribute is defined on the child elements.

```
@Bind( expression="table:table-row", namespaceAlias = {"table"},
namespaceUris = {"urn:oasis:names:tc:opendocument:xmlns:table:1.0"},
collectionBindStrategy = SerializingMode.BIND_COLLECTION_BY_POSITION)
```

```
public class Row implements Serializable {

@Bind(collectionBindStrategy =
SerializingMode.BIND_COLLECTION_BY_POSITION,namespaceAlias = {"table"},
namespaceUris = {"urn:oasis:names:tc:opendocument:xmlns:table:1.0"})

private List Cell> cells = new ArrayList Cell>();
```

- Append new elements as last (BIND_COLLECTION_APPEND_LAST).

This strategy assumes that the bind is made by key, but new elements are appended at the end of the xml file.

- Insert new elements as first (BIND_COLLECTION_APPEND_FIRST).

This strategy assumes that the bind is made by key, but new elements are insered at the beginning of the xml file.

- Full collection reset (BIND_COLLECTION_FULL_RESET).

This strategy is equivalent to traditional object binding. It erases all the data related to the collection and serializes the information taking the object information as an argument.

Bind Annotation Reference.

The Bind annotation reference has the fo

import net.vionta.xml.xesta.bind.annotation.Bind;

- expression The XPath mapping expression that points to the selected document nodes.
- key Indicates if the attribute identifies the node element. It is only considered when the collection strategy is equal to SerializingMode. BIND_COLLECTION_BY_KEY.
- classNames [String | Default : null | Mandatory : false] The class names of a multiple collection. An element list can containt more than one single element type. Xesta looks for the mappings in all of the classes to determine the collection elements. *This feature is under development*
- serializingMode
- deserializingMode
- collectionBindStrategy
- collectionDeleteUnmatched
- namespaceAlias [String | Default : null | Mandatory : false] A list of the name space alias.
- namespaceUris [String | Default : null | Mandatory : false] A list of the name space uris.
- auto If true the element name or expression is calculated from the java property name.
- DeserializingMode
  - FAIL_ON_NOT_EXISTING : 1
  - WARN_ON_NOT_EXISTING : 2
  - AVOID_ON_NOT_EXISTING : 3
- SerializingMode
  - FAIL_ON_NOT_EXISTING : 1
  - CREATE_ON_NOT_EXISTING : 2
  - SKIP_ON_NOT_EXISTING : 3
  - BIND_COLLECTION_BY_KEY : 1
  - BIND_COLLECTION_BY_POSITION : 2
  - BIND_COLLECTION_APPEND_LAST : 3
  - BIND_COLLECTION_APPEND_FIRST : 4
  - BIND_COLLECTION_FULL_RESET : 5
  - COLLECTION_DONT_DELETE_UNMATCHED : 1

- COLLECTION_DELETE_UNMATCHED : 2

Repository types

Xesta access to documents are wrapped with the repository abstraction. In order to retrieve or update a document you need to choose from one of the repository types, provide the configuration parameters, usually the path or paths.

Once the repository is configured (its path or paths are provided) you request the repository to retrieve or update the document for you.

Path Adjustment.

The repository paths (in the supported cases) can be adjusted based on the object properties. In order to configure the adjustment the property name (separated with dots between objects) must be defined inside the key { } markers.

File/Folder based repository.

The simplest possible repository type with a base path and a file path. The base path is not adjusted based on the object properties, while the file path can be adjusted with the {} keys.

```
FilePathRepository repo = new
 FilePathRepository(CommonTest.getBasePath()+"cases/iso20022/
GlobalPayment.xml");
GlobalPayment globalPayment = new GlobalPayment();
globalPayment = repo.load(globalPayment);
```

Classpath repository.

A repository that retrieves files from the application classpath. This repository does not allow to create new files, also some files may be available for reading but not accesible for writing. The file path is defined with simple file path conventions separating folders with the slash "/" operator.

```
  ClassPathRepository repo = new ClassPathRepository().setPath("log4j.xml");

  LogLevelSimple log = repo.load(new LogLevelSimple());
  log.setLevel("DEBUG");
  repo.persist(log);
```

Mind that, depending on the running environment, the document path may or may not be accesible using the file system.

REST based repository.

A repository that retrieves and stores the files from a REST service. The file path can also be adjusted via the object data using the {} keys on the path.

```
** Note: service authentication not yet developed. **
```

Zip file based repository.

The zip file repository is similar to the file repository but acts on files that are placed inside zip files. Both the zipfile and the content file can be ajusted via the object properties.

Zip file names are not required to end in ".zip". Zip repository can be used for odt, ods, wars, etc.

```
String zipFilePattern = CommonTest.getBasePath()+"openDocument/text/
fictional-report.odt";
ZipArchiveFileRepository repo =
         new ZipArchiveFileRepository(
    zipFilePattern,"content.xml");
Document1 doc1 = new Document1();
doc1 = (Document1) repo.load(doc1);
```

Abstract repository.

An abstract implementation that supports extensibility in other cases.

Still not available.

```
 ** Note: Functionality in development. **
```

Consuming a remote RDF from the DBpedia with Xesta

One of the several serialization formats used with rdf is Xml. You may be more prone to use turtle, owl or other formats when handling rdf information, but when it comes to integrate data Xml is one of the best choices.

There are lots of data resources on the web, like the Dbpedia, that can be consumed from applications without over-engineer your data model. Consuming the Dbpedia with Xesta is almost trivial for a java developer.

Introducing the resource sample

We are going to query an airport information, taking the Vigo Airport as a sample. The url request would look like the following.

curl 'https://dbpedia.org/data/Vigo–Peinador_Airport.rdf'

The returned resource should look like:

?xml version="1.0" encoding="utf-8" ?> rdf:RDFxmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"xmlns:owl="http://www.w3.org/2002/07/owl#"xmlns:foaf="http://xmlns.com/foaf/0.1/"xmlns:dct="http://purl.org/dc/terms/"xmlns:dbo="http://dbpedia.org/ontology/"xmlns:dbp="http://dbpedia.org/property/"xmlns:geo="http://www.w3.org/2003/01/geo/wgs84_pos#"xmlns:prov="http://www.w3.org/ns/prov#"xmlns:georss="http://www.georss.org/georss/"xmlns:gold="http://purl.org/linguistics/gold/" > rdf:Description rdf:about="http://dbpedia.org/resource/List_of_airports_by_ICAO_code:_L"> dbo:wikiPageWikiLink rdf:resource="http://dbpedia.org/resource/Vigo–Peinador_Airport" /> /rdf:Description> rdf:Description rdf:about="http://dbpedia.org/resource/VG-20"> dbo:wikiPageWikiLink rdf:resource="http://dbpedia.org/resource/Vigo–Peinador_Airport" /> dbo:routeStart rdf:resource="http://dbpedia.org/resource/Vigo–Peinador_Airport" /> dbp:terminusA rdf:resource="http://dbpedia.org/resource/Vigo–Peinador_Airport" /> /rdf:Description> rdf:Description rdf:about="http://dbpedia.org/resource/List_of_airports_by_IATA_airport_code:_V"> dbo:wikiPageWikiLink rdf:resource="http://dbpedia.org/resource/Vigo–Peinador_Airport" /> /rdf:Description> rdf:Description rdf:about="http://dbpedia.org/resource/Tenerife_North–Ciudad_de_La_Laguna_Airport"> dbo:wikiPageWikiLink rdf:resource="http://dbpedia.org/resource/Vigo–Peinador_Airport" /> /rdf:Description> rdf:Description rdf:about="http://dbpedia.org/resource/Valencia_Airport"> dbo:wikiPageWikiLink rdf:resource="http://dbpedia.org/resource/Vigo–Peinador_Airport" /> /rdf:Description> rdf:Description rdf:about="http://dbpedia.org/

resource/List_of_Air_Europa_destinations"> dbo:wikiPageWikiLink rdf:resource="http://dbpedia.org/resource/Vigo–Peinador_Airport" /> /rdf:Description> rdf:Description rdf:about="http://dbpedia.org/resource/Juan_Carlos_I"> dbo:wikiPageWikiLink rdf:resource="http://dbpedia.org/resource/Vigo–Peinador_Airport" /> /rdf:Description>...

Annotating the Pojo with mappings.

In order to define from where xml elements our information is taken from, we will create and map a Java POJO. Make sure you have defined the Xesta dependency or added the xesta jar to your path.

<dependency><groupId>net.vionta.xml</groupId><artifactId>xesta</artifactId><version>1.0.1</version></dependency>

We define the property to xml mappings using the Bind annotation. For each class or mapped property, we will add the Bind annotation and add the XPath expression to describe the mapping. The property for the XPath expression is unsurprisingly called "expression".

The significative parts of the java code should look like the code shown below. Regular Setters and Getters should be added for each mapped property.

@Bind(expression="/*:RDF")public class Airport implements Serializable {

public Airport() {}

private String key;

@Bind(expression=".//dbp:name/text()", namespaceAlias= {"dbp"},namespaceUris= {"http://dbpedia.org/property/"})private String name;

@Bind(expression=".//*:label/text()")private String localName;

```
@Bind(expression=".//dbo:runwayLength[1]",
 namespaceAlias = {"dbo"},
 namespaceUris = {"http://dbpedia.org/ontology/"})
```

private String runwayLenght;

@Bind(expression=".//*:stat1Data")private Integer passengers;

@Bind(expression=".//*:stat2Data")private String passengersEvolution;

@Bind(expression=".//*:stat3Data")private String movements;

@Bind(expression=".//*:stat4Data")private Integer cargo;

@Bind(expression=".//georss:point",namespaceAlias = {"georss"},namespaceUris = {"http://www.georss.org/georss/"})private String position;

@Bind(expression=".//dbp:iata",namespaceAlias = {"dbp"},namespaceUris = {"http://dbpedia.org/property/"})private String iataCode;

@Bind(expression=".//dbo:icaoLocationIdentifier",namespaceAlias = {"dbo"},namespaceUris = {"http://dbpedia.org/ontology/"})private String icaoCode;

```
//..
```

You can generate the required setters and getters using your IDE or check the full code sample at the samples repository.

Querying the resource

The code needed for quering the resource is also really simple. Just create an HttpRepository object instance. We are also going to use the "key" property, from the mapping bean, to allow to switch between airports. You may notice the key placeholder in the path param.

```
HttpRepository repo = new HttpRepository();repo.setDocumentPath("https://dbpedia.org/data/.rdf");
```

We create an Airport object and set the key to the desired airport.

```
Airport testAirport = new Airport();testAirport.setKey("Vigo–Peinador_Airport");
```

We can now load the airport information from the wikipedia.

```
testAirport = repo.load(testAirport);
```

You should be able to check the information.

```
13:43:24,970 DEBUG  Airport
```

At this point, you can play with the mappings, try to retrieve different values or query different resources.

Querying different rdf resources with arq.

Rdf resources are usually queried with SPARQLE queries. A simple way to use them is to use the jena arq program. An arq query can be executed to extract a tabular version of the data extracted from an SPARQLE query on a rdf resource. The result can be exported to an xml file.

```
$ apache-jena-4.9.0/bin/arq --query queries/Practices.sparqle --data ./../notas-metodologias-definiciones-turtle.ttl --out XML > results/Practice-Types.xml
```

The result xml output can be stored as an xml file and map a Xesta bean to use the information. In this case we would be using a File repository. You may also directly use Apache Jena.

References

- Dbpedia https://www.dbpedia.org
- Apache Jena https://jena.apache.org
- Apache Maven https://maven.apache.org
- Xesta dependency https://mvnrepository.com/artifact/net.vionta.xml/xesta
- Xesta https://vionta.net/vionta/community/xesta.html

Consuming a remote Xml file with Xesta

There are lots of data resources available as Xml on the web. Xml is a gread data serialization format when it comes to data integration. We are going to show how web resources can be easily queried and used from Java using Xesta.

We are going to use the spanish weather service and the city of Vigo as sample. The weather predictions from Aemet, the spanish weather service, is available as Xml on the web.

Introducing the resource sample

We are going to query the Vigo weather prediction. The city id could be parametrized using one of the pojo properties. The end url should look like the following.

```
curl 'https://www.aemet.es/xml/municipios/localidad_36057.xml'
```

The curl request answer should look like the following:

```
 root xmlns:xsd="https://www.w3.org/2001/XMLSchema" xmlns:xsi="https://www.w3.org/2001/
XMLSchema-instance" id="36057" version="1.0" xsi:noNamespaceSchemaLocation="https://
www.aemet.es/xsd/localidades.xsd">  !-- https://www.aemet.es/xml/municipios/localidad_36057.xml
-->  origen>  productor>Agencia Estatal de Meteorología - AEMET. Gobierno de España /
productor>  web>https://www.aemet.es /web>  enlace>https://www.aemet.es/es/eltiempo/
prediccion/municipios/vigo-id36057 /enlace>  language>es /language>  copyright>© AEMET.
Autorizado el uso de la información y su reproducción citando a AEMET como autora de la
misma. /copyright>  nota_legal>https://www.aemet.es/es/nota_legal /nota_legal>  /origen>
 elaborado>2026-10-14T08:55:08 /elaborado>  nombre>Vigo /nombre>  provincia>Pontevedra /
provincia>  prediccion>  dia fecha="2026-10-14">  prob_precipitacion periodo="00-24"/>
 prob_precipitacion periodo="00-12"/>  prob_precipitacion periodo="12-24">0 /prob_precipitacion>
 prob_precipitacion periodo="00-06"/>  prob_precipitacion periodo="06-12">0 /prob_precipitacion>
 prob_precipitacion periodo="12-18">0 /prob_precipitacion>  prob_precipitacion periodo="18-24">0 /
prob_precipitacion>  cota_nieve_prov periodo="00-24"/>  cota_nieve_prov periodo="00-12"/>
 cota_nieve_prov periodo="12-24"/>  cota_nieve_prov periodo="00-06"/>

...
```

Annotating the Pojo with mappings

We will create and configure two Java POJOs, one for the location prediction and one for each of the available days. Make sure you have defined the Xesta dependency on the project pom or added the xesta jar to your path.

```
<dependency><groupId>net.vionta.xml</groupId><artifactId>xesta</artifactId><version>1.0.1</
version></dependency>
```

We have to add the Bind annotation for the class and each property. In each annotation we should add the XPath expression to the desired Xml node. The property for the XPath expression is unsurprisingly called "expression". We have added the source xml data sample on the property comments for reference.

The significative parts of the java code for the location should look like the code shown below. Regular Setters and Getters should be added for each mapped property.

```
@Bind(expression = "/root") public class WeatherLocation implements Serializable {

public WeatherLocation() {}

@Bind(expression = "nombre") String locationName ;

@Bind(expression = "@id") String locationCode ;

@Bind(expression="prediccion/dia") private List<DayPrediction> nextDays = new
ArrayList<DayPrediction>();

public String getLocationName() { return locationName; }

public void setLocationName(String locationName)

    //..
```

After the fields are added and configured you can generate the settes and getters with your IDE. The DayPrediction java code may look like the following sample:

```
@Bind public class DayPrediction implements Serializable {

public DayPrediction() {}

@Bind(expression="@fecha") private String date ;

@Bind(expression="cota_nieve_provtext()!=0") private Integer snow;

@Bind(expression="prob_precipitaciontext()!=0") private Integer rain;
```

```
@Bind(expression="estado_cielo") private String clouds;

@Bind(expression="max(viento/velocidad/text())") private Integer wind;

...
```

You may notice that we don't need to provide an XPath expression for the DayPrediction object inside the Class Bind annotation, as it has been provided on the collection mapping.

Querying the resource

The code needed for quering the resource is also quite simple. Just create an HttpRepository object. We are going to use the key property of the bean to allow to switch between airports. You will notice the key placeholder in the path param.

We can create an http repository object to query the resource with a code like the following.

```
HttpRepository repo = new HttpRepository(); repo.setDocumentPath("https://www.aemet.es/xml/
municipios/localidad_.xml");
```

Then we can create a parametrizable weather location object with the following code:

```
WeatherLocation weatherLocation = new WeatherLocation(); weatherLocation.setLocationCode("36057");
```

To retrieve the weather information we just need to load the object from the repository.

```
weatherLocation = repo.load(weatherLocation); log.debug("Gotted weather :");
log.debug(weatherLocation.toString());
```

The end result should look similar to the following one:

```
15:20:21,862 DEBUG  WeatherLocation [locationName=Vigo, locationCode=36057,
nextDays=[DayPrediction , DayPrediction , DayPrediction , DayPrediction , DayPrediction ,
DayPrediction , DayPrediction ]]
```

At this point, you can play with the mappings, try to retrieve different values or query different resources.

The full code can be checked at the xesta samples repository on Github.

References

- Dbpedia https://www.aemet.es
- Apache Maven: https://maven.apache.org
- Xesta dependency: https://mvnrepository.com/artifact/net.vionta.xml/xesta
- Xesta: https://vionta.net/vionta/community/xesta.html
- Xesta samples at Github: https://github.com/vionta/xesta-samples

Status

Xesta has been developed as an experiment and a proposal for XML Prague 2026. First scheduled presentation will take place at the Prague Economics University on June 6. At this stage, Xesta could be considered as an alpha version. Anyhow, remarkable number of tests has been performed an executed.

Changelog

- 2026-09-03 Released documentation version 1.1. Documentation and release updated.

- 2026-06-08 Fento is renamed as Xesta. Due to a possible coincidence with another component the Fento Xml Binding component has been renamed as Xesta.

Acknowledgements.

- Saxonica The XPath heavy lifting is performed by the mighty tool. www.saxonica.com

References.

- XML Prague presentation The tool, named Fento at the time, was presented at Prague Xml in 2026. https://www.xmlprague.cz/day3-2026/